



Bapuji Educational Association®
Bapuji Institute of Engineering and Technology, Davanagere
An Autonomous Institute Affiliated to VTU
Department of Computer Science and Engineering

Course Title	Database Management Systems	Course Code	BCSPCC403
Course Instructor	Dr. Gururaj T, Prof.Hemashree ,Prof.Shwetha G., Prof.Roopaa & Prof. Santhosh P	Semester-Section	IV
IA Test No.	II	Max. Marks	30
Date	11-06-2026	Time	9.30 to 10.45

Course Outcome Statements: After the successful completion of the course, the students will be able to	
CO1	Conceptualize data modelling concepts to draw ER-diagrams for the given scenarios.
CO2	Create efficient queries to retrieve data from the database considered.
CO3	Apply the database normalization concepts to devise efficient databases for the given real time problems.
CO4	Analyse the importance of Transaction processing and concurrency control in efficient database management.
CO5	Differentiate between DBMS and NoSql.

Note: Answer any one full question from each Part.					
Q. No.	Question		Marks	RBT Level	CO
Part A					
1	A	Consider the schema for Employee Database: EMP(<u>EmpID</u> , EmpName, Salary, DeptID) DEPT(<u>DeptID</u> , DeptName, ManagerID) Write SQL queries to: i. Retrieve the employee name and department name of all employees. ii. List the names of employees working in the "CSE" department. iii. Display employee names along with their salaries and department names.	6	L3	3
OR					
2	A	Consider the schema for Students Database: STUDENT(<u>USN</u> , SName, Sem, DeptID) DEPARTMENT(<u>DeptID</u> , DeptName, HOD) COURSE(<u>CourseID</u> , CourseName, DeptID) Write SQL queries to: i. Display student names along with their semester and department names. ii. List all students and the courses offered by their department. iii. Display student names along with the HOD of their department.	6	L3	3
Part B					
3	A	Define Join Dependency and Fifth Normal Form (5NF). Illustrate with an example.	6	L2	4
	B	What is transaction processing? Explain the key transaction and system concepts, such as recovery and concurrency control.	6	L2	4
OR					
4	A	What is Boyce-Codd Normal Form (BCNF)? Differentiate it from 3NF with a suitable example where a relation is in 3NF but not in BCNF.	6	L2	4
	B	Describe transaction support in SQL, including commands like COMMIT, ROLLBACK and SAVEPOINT.	6	L2	4
Part C					
5	A	Discuss the concept of the Two-Phase Locking (2PL) protocol used in database concurrency control.	6	L2	5
	B	Explain the primary CRUD (Create, Read, Update, Delete) operations in MongoDB with suitable examples.	6	L2	5



Bapuji Educational Association®
Bapuji Institute of Engineering and Technology, Davanagere
An Autonomous Institute Affiliated to VTU
Department of Computer Science and Engineering

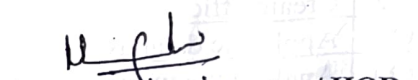
Note: Answer any one full question from each Part.							
Q. No.	Question			Marks	RBT Level	CO	
OR							
6	A	Explain the concepts of deadlock and starvation in database systems.			6	L2	5
	B	Describe the CAP theorem in distributed database systems. Which of the three properties are most important in NOSQL systems?			6	L2	5

RBT (Revised Bloom's Taxonomy) Levels : Cognitive Domain					
L1 : Remembering	L2 : Understanding	L3 : Applying	L4 : Analysing	L5 : Evaluating	L6 : Creating


Course Instructor


Course Coordinator


DAAC


Program Coordinator / HOD



Bapuji Educational Association (Regd.)

Bapuji Institute of Engineering and Technology, Davanagere - 04

Department of Computer Science and Engineering (CS and allied Branches)

Course Title & Code:	Database Management Systems [BCSPCC403]	Test No.:	2
Semester:	4th Sem	Date:	11-06-2026
Course Instructor	Dr. Gururaj T, Prof. Hemashree, Prog. Shwetha G, Prof. Roopa, Prof. Santhosha P	Marks	30

Qn.	Scheme	Marks
	[PART-A]	
1 A. =	i. Select E. Empname, D. DeptName From EMP E Join Dept D on E. DeptID = D. DeptID; →	(2M)
	ii. Select E. Empname From Emp E Join Dept D on E. DeptID = D. DeptID Where D. DeptName = 'CSE'; →	(2M)
	iii. Select E. EmpName, E. Salary, D. DeptName From Emp E Join Dept D on E. DeptID = D. DeptID. →	(2M)
		Total (6M)

Qn.	Scheme [OR]	Marks
2A	i> <pre> Select S.SName, S.Sem, D.DeptName From Student S Join Department D on S.DeptID = D.DeptID;</pre> ii> <pre> Select S.SName, C.CourseName From Student S Join Course C on S.DeptID = C.DeptID;</pre> iii> <pre> Select S.SName, D.HOD From Student S Join Department D on S.DeptID = D.DeptID</pre>	→ 2M — 2M — 2M Total (6M)

Qn.	Scheme	Marks																											
3a	[PART-B] <u>Join Dependency</u>:- It is denoted by $JD(R_1, R_2 \dots R_n)$, Specified on Relation Schema R, specifies a constraints on the states r of R. The constraint states that every legal state r of R should have a nonadditive join decomposition into $R_1, R_2 \dots R_n$. Hence, for every such r we have $* (\pi_{R_1}(r), \pi_{R_2}(r), \dots, \pi_{R_n}(r)) = r.$ $JD(R_1, R_2) = (R_1 \cap R_2) \twoheadrightarrow (R_1 - R_2)$ <u>Fifth Normal Form (5NF)</u> A relation Schema R is in 5NF (Project join normal Form) with respect to a set of F, functional, multivalued, and join Dependencies if, for every non-trivial join Dependency $JD(R_1, R_2 \dots R_n)$ in F^+, for every R_i is a Superkey of R. <u>Ex:-</u> Relation (Faculty) <table border="1" style="margin: 10px auto; border-collapse: collapse;"> <tr> <th>Subject</th> <th>faculty</th> <th>year</th> </tr> <tr> <td>C</td> <td>Sai</td> <td>1</td> </tr> <tr> <td>DBMS</td> <td>Ajith</td> <td>3</td> </tr> </table> (Sub-fac) <table border="1" style="border-collapse: collapse;"> <tr> <th>Subject</th> <th>faculty</th> </tr> <tr> <td>C</td> <td>Sai</td> </tr> <tr> <td>DBMS</td> <td>Ajith</td> </tr> </table> (Fac-year) <table border="1" style="border-collapse: collapse;"> <tr> <th>faculty</th> <th>year</th> </tr> <tr> <td>Sai</td> <td>1</td> </tr> <tr> <td>Ajith</td> <td>3</td> </tr> </table> (Sub-year) <table border="1" style="border-collapse: collapse;"> <tr> <th>Subject</th> <th>year</th> </tr> <tr> <td>C</td> <td>1</td> </tr> <tr> <td>DBMS</td> <td>3</td> </tr> </table>	Subject	faculty	year	C	Sai	1	DBMS	Ajith	3	Subject	faculty	C	Sai	DBMS	Ajith	faculty	year	Sai	1	Ajith	3	Subject	year	C	1	DBMS	3	3M 3M Total 6M
Subject	faculty	year																											
C	Sai	1																											
DBMS	Ajith	3																											
Subject	faculty																												
C	Sai																												
DBMS	Ajith																												
faculty	year																												
Sai	1																												
Ajith	3																												
Subject	year																												
C	1																												
DBMS	3																												

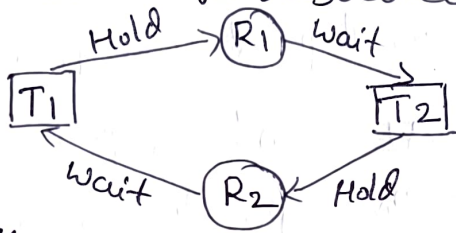
Qn.	Scheme	Marks
3b	<u>Transaction processing</u> :- Transaction processing are systems with large databases and hundreds of concurrent users executing databases transaction. <u>Key transaction and system concepts</u> *? BEGIN-TRANSACTION *? READ & WRITE *? END-TRANSACTION *? COMMIT-TRANSACTION *? ROLLBACK (or ABORT) Explain. <u>System Log.</u> *? [Start-transaction, T] *? [write-item, T, x, old-value, new-value] *? [read-item, T, x] *? [commit, T] *? [abort, T] <pre> graph LR Start[Begin transaction] --> Active((Active)) Active -- "End transaction" --> PC((Partially committed)) Active -- "Abort" --> Failed((Failed)) PC -- "commit" --> Committed((Committed)) PC -- "failed" --> Failed Failed --> Terminated((Terminated)) Committed --> Terminated </pre> fig: state transition Diagram for transaction execution	6M

Qn.	Scheme	Marks						
4A	<u>[OR]</u> <u>Boyce-Codd Normal Form (BCNF)</u> *BCNF is an extension to the 3NF, it is also known as 3.5NF. → A relation schema R is in BCNF if whenever a non trivial functional dependency $X \rightarrow A$ holds in R, then X is a super key of R. <u>Example</u> FD1 : {Student, Course} → Instructor FD2 : Instructor → Course. <u>Solution</u> S.K = $(\text{Student, Course, Instructor})^+ = \{\text{Student, Course, Instructor}\}$ C.K $(\text{Student, Course})^+ = \{\text{Student, Course, Instructor}\}$ R.H.S of FD2. $(\text{Student, Instructor})^+ = \{\text{Student, Instructor, Course}\}$ Student → A R Course → B Instructor → C <table border="1" style="margin-left: auto; margin-right: auto;"> <tr> <td>A</td><td>B</td><td>C</td></tr> <tr> <td></td><td></td><td></td></tr> </table> Table is in 3NF but not in BCNF. <u>Decomposition of Relation R to BCNF.</u> $R_1 (\text{Student, Instructor}) \ \& \ R_2 (\text{Student, Course})$ $R_1 (\text{Course, Instructor}) \ \& \ R_2 (\text{Course, Student})$ $R_1 (\text{Instructor, Course}) \ \& \ R_2 (\text{Instructor, Student})$	A	B	C				6M
A	B	C						

Qn.	Scheme	Marks
4b	<u>Transaction support in SQL</u> Transaction is a group of SQL operations executed as a single unit of work, it ensures data integrity by following principles of ACID (Atomicity, Consistency, Isolation, Durability) <u>Transaction Commands</u> *> <u>Commit</u> :- Save all changes made during the transaction permanently to the database. <u>Ex:-</u> COMMIT; *> <u>ROLLBACK</u> :- undoes all changes made in the current transaction since the last COMMIT. <u>Ex:-</u> ROLLBACK; *> <u>SAVEPOINT</u> :- creates a point within a transaction to which you can later rollback. <u>Ex:-</u> <u>SAVEPOINT</u> SP1; BEGIN Transaction UPDATE Accounty SET Balance = Balance - 1000 WHERE Accountno = 101; SAVEPOINT SP1; UPDATE Accounty SET Balance = Balance + 1000 WHERE Accountno = 102; COMMIT;	2M 4M Total 6M

Qn.	Scheme	Marks																											
	[PART-C]																												
5A	<u>Two phase Locking</u> It is one of the concurrency control technique. It requires both locks & unlocks. 2PL is divided into 2 Phases 1) growing phase 2) Shrinking phase. → <u>growing phase</u> :- Transaction obtained only locks the transaction, but not release the transaction → <u>Shrinking phase</u> :- Locks are released but no new locks are required. These 2 are having lock point. <u>Example</u> <table border="0"> <tr> <td>T₁</td> <td>T₂</td> <td></td> </tr> <tr> <td>L₁(A)</td> <td></td> <td>1) Conservative (Stable)</td> </tr> <tr> <td>R(A)</td> <td></td> <td>2) Strict 2PL</td> </tr> <tr> <td>A = A + 100</td> <td></td> <td>3) Rigorous 2PL. } →</td> </tr> <tr> <td>W(A)</td> <td></td> <td></td> </tr> <tr> <td>L₁(B)</td> <td></td> <td>⇒ <u>conservative 2PL</u>.</td> </tr> <tr> <td>unlock(A)</td> <td></td> <td>Locks all the data item before transaction. After completion all operation then only release.</td> </tr> <tr> <td>⋮</td> <td></td> <td></td> </tr> <tr> <td>⋮</td> <td></td> <td></td> </tr> </table> ⇒ <u>Strict 2PL</u> All exclusive locks should hold until commit / Abort. ⇒ <u>Rigorous 2PL</u> All shared and exclusive locks should hold until commit / Abort.	T ₁	T ₂		L ₁ (A)		1) Conservative (Stable)	R(A)		2) Strict 2PL	A = A + 100		3) Rigorous 2PL. } →	W(A)			L ₁ (B)		⇒ <u>conservative 2PL</u> .	unlock(A)		Locks all the data item before transaction. After completion all operation then only release.	⋮			⋮			3M 3M Total 6M
T ₁	T ₂																												
L ₁ (A)		1) Conservative (Stable)																											
R(A)		2) Strict 2PL																											
A = A + 100		3) Rigorous 2PL. } →																											
W(A)																													
L ₁ (B)		⇒ <u>conservative 2PL</u> .																											
unlock(A)		Locks all the data item before transaction. After completion all operation then only release.																											
⋮																													
⋮																													

Qn.	Scheme	Marks
5B	<u>MongoDB</u> :- Store the data in documents similar to JSON (JavaScript object notation) 2M → Document contains pairs of field & value. <u>CRUD operations in MongoDB</u> C - Create R - Read U - update D - Delete 1) <u>Create</u> :- use COMPANY → Database creation db.createCollection("Employee") 2) <u>R-Insert Documents</u> db.Employee.insertOne({name: 'ABC', Branch: 'AIML'}) db.Employee.insertMany([{name: 'XYZ', Branch: 'CS'}, {name: 'ABC'}]) 3) <u>Read</u> :- Read the documents from Employee collection 4M db.Employee.find() 4) <u>update</u> db.Employee.updateOne({name: 'ABC', Branch: 'IS'}) db.Employee.updateMany({name: 'ABC', Branch: 'IS'}) 5) <u>Delete</u> :- delete one or more documents from Employee collection Total 6M db.Employee.deleteOne({name: 'ABC'}) db.Employee.deleteMany({name: 'ABC'})	

Qn.	Scheme	Marks
6a	[OR] <u>Deadlock</u> occurs when two or more transactions are waiting indefinitely for each other to release resources,  <u>Necessary conditions for Deadlock occur</u> 1 > mutual Exclusion 2 > Hold & wait 3 > NO-preemption 4 > Circular wait <u>Deadlock Avoidance/prevention</u> 1 > wait and die technique 2 > Wound and wait technique <u>Deadlock Detection Technique</u> 1 > wait for Graph 2 > Resource Allocation Graph. <u>Starvation</u> starvation occurs when a transaction wait for a resource for an unbounded period because other transactions can continuously get priority.	Total 6M

Qn.	Scheme	Marks
6b	<u>CAP</u> Theorem states that a distributed database can provide only two of the three properties: consistency, Availability and partition Tolerance. → Most NOSQL databases prioritize Availability and partition Tolerance to ensure continuous service and fault tolerance in distributed system databases. <u>Properties</u> <u>C</u> (Consistency) :- All nodes see the same data at same time. <u>A</u> (Availability) :- Every request receive a response, even if some nodes are fail. <u>P</u> (Partition Tolerance) :- The system continues to operate despite network communication failure between nodes.	→ 3M → 3M Total (6M)

[Signature]
Course coordinators

[Signature]
DAAC
for

[Signature]
Program coordinator
(HOD. CS & Engg)

[Signature]
BOE Chairman